

Instituto Tecnológico de Costa Rica

Ingeniería en Computación

Simulador de Gestión de Procesos

Mini PC — Documentación Final

Curso: Principios de Sistemas Operativos

Profesor: Cristian Campos

Estudiante: Jordano Escalante

Período: Primer Semestre

2026

Índice General

1. Descripción del Proyecto

- ↳ Stack Tecnológico
 - ↳ Arquitectura General
-

2. Inventario de Paquetes

- ↳ assembler
 - ↳ clock
 - ↳ cpu
 - ↳ fx
 - ↳ interrupt
 - ↳ io
 - ↳ kernel
 - ↳ memory
 - ↳ model
 - ↳ process
 - ↳ scheduler
 - ↳ stats
 - ↳ storage
-

3. Clases Clave — Diagrama UML

- ↳ PCB
 - ↳ Kernel
 - ↳ CPU
 - ↳ Dispatcher
 - ↳ SimuladorController
 - ↳ 3.1 Cardinalidad de Clases
 - ↳ 3.2 Cardinalidad de Paquetes
 - ↳ 3.2.1 Vista Estructural
 - ↳ 3.2.2 Flujo por Tick
 - ↳ 3.3 Especificación de Clases
-

4. Pipeline de Ejecución

- ↳ 4.1 Ciclo Tick (10 etapas)
 - ↳ 4.2 Secuencia de Carga
 - ↳ 4.3 Secuencia de Tick
-

5. Gestión de Memoria

- ↳ 5.1 First-Fit

6. Interrupciones y E/S

- ↳ Latencias
- ↳ Tipos de interrupción
- ↳ Flujo INT 21H

7. Manejo de Errores

- ↳ 7.1 Errores de parse
- ↳ 7.2 Errores de runtime

8. Métricas de Rendimiento

9. Conjunto de Instrucciones ASM

10. Estados del Proceso

11. Manual de Usuario

- ↳ 11.1 Inicio rápido
- ↳ 11.2 Flujo operativo
- ↳ 11.3 Interpretación de paneles
- ↳ 11.4 Incidencias frecuentes

12. Manual Técnico

- ↳ 12.1 Arquitectura y responsabilidades
- ↳ 12.2 Configuración externa (JSON)
- ↳ 12.3 Contratos de mantenimiento
- ↳ 12.4 Pruebas técnicas sugeridas

13. Objetivos Alcanzados

14. Bibliografía

1. Descripción del Proyecto

`SimuladorMiniPC` es una aplicación de escritorio educativa que implementa un simulador de sistema operativo de tiempo discreto. Modela los mecanismos fundamentales de gestión de procesos: ciclo de vida, planificación de CPU, asignación de memoria, interrupciones y dispositivos de E/S.

Concepto simulado	Mecanismo
Multiprogramación	Hasta 5 procesos concurrentes en memoria

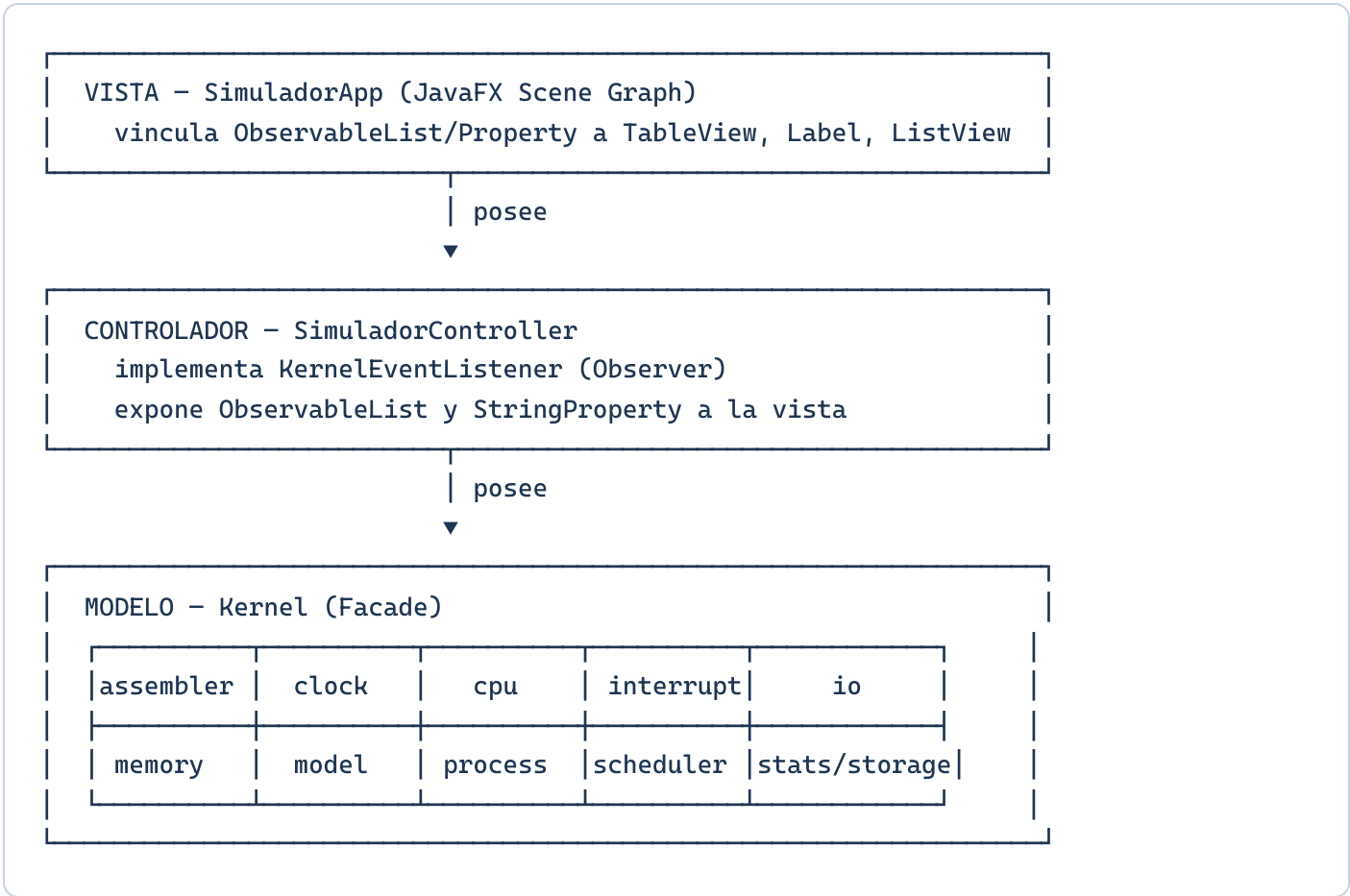
Concepto simulado	Mecanismo
Estados de proceso	7 estados: NEW → READY → RUNNING → BLOCKED → TERMINATED
Planificación	Política activa actual: FCFS (infraestructura extensible para otras políticas)
Gestión de memoria	RAM 700 celdas, first-fit, área del SO (celdas 0–19)
Memoria virtual	Traducción base+offset por proceso; validación de offsets
Interrupciones	INT 20H, INT 10H, INT 09H, INT 21H
E/S con latencias	Teclado (MAX_VALUE), Pantalla (1 tick), Disco (5 ticks)
Estadísticas	Utilización CPU, throughput, tiempos de espera/retorno/respuesta

Stack Tecnológico

Componente	Versión	Rol
Java	25	Lenguaje principal
JavaFX	24	Scene Graph, ObservableList, bindings, Timeline, TableView
NetBeans Ant	—	build.xml, nbproject/ — compilación y empaquetado
JSON (sin libs)	—	MemoryConfig, DiskConfig y AssemblerConfig parsean con regex propio
PlantUML	—	Diagramas UML en <code>documentacion/diagramas/</code>

Arquitectura General

El sistema sigue el patrón **MVC** con extensión **Observer** para desacoplar el kernel de la UI:



2. Inventario de Paquetes

Esta sección presenta el mapa modular del simulador desde una perspectiva de arquitectura. Su propósito es explicar cómo se distribuyen responsabilidades por paquete para reforzar mantenibilidad, trazabilidad de cambios y estrategia de pruebas.

Total: **14 paquetes**, **43 clases Java**.

simuladorminipc (raíz)

├─ assembler

├─ clock

├─ cpu

├─ fx

├─ interrupt

├─ io

├─ kernel

├─ memory

├─ model

├─ process

├─ scheduler

├─ stats

└─ storage

← parser léxico/sintáctico del ASM propio

← reloj global monotónico

← CPU virtual + Dispatcher

← capa de presentación JavaFX

← cola FIFO de interrupciones

← dispositivos de E/S con latencias

← núcleo orquestador (Fachada)

← RAM, first-fit, traducción virtual, area del SO

← PCB, RegisterSet, ProcessState, IORequest

← admisión, colas y transiciones de estado

← política activa FCFS (otras políticas en código)

← métricas de rendimiento

← disco secundario simulado

simuladorminipc.assembler

Análisis léxico y sintáctico. Transforma archivos `.asm` en listas de objetos `Instruction` listos para ejecutar.

Clase	Responsabilidad
<code>Assembler</code>	Clase utilitaria estática. <code>parse(File)</code> tokeniza y valida cada línea. Límite de instrucciones configurable.
<code>AssemblerConfig</code>	Lee <code>assembler-config.json</code> del classpath. Expone <code>maxInstructions</code> (default 80; 0 = sin límite).
<code>Instruction</code>	Objeto de valor inmutable: tipo, operandos, línea fuente, peso.
<code>InstructionType</code>	Enum de 19 tipos con su peso de CPU.

simuladorminipc.clock

Clase	Responsabilidad
<code>SystemClock</code>	Contador <code>long</code> monotónico. <code>tick()</code> , <code>reset()</code> , <code>getCurrentTick()</code> .

simuladorminipc.cpu

Clase	Responsabilidad
CPU	Unidad de procesamiento. <code>executeCycle()</code> descuenta peso y llama a <code>applyInstruction()</code> .
Dispatcher	Cambios de contexto formales: <code>dispatch(PCB, CPU)</code> y <code>preempt(CPU)</code> . Contabiliza switches.
CycleEvent	Valor inmutable resultado de un ciclo. 7 fábricas estáticas.
CycleResult	Enum: NORMAL, PROCESS_FINISHED, SCREEN_OUTPUT, KEYBOARD_INPUT, FILE_OPERATION, IDLE, ERROR.

simuladorminipc.fx

Clase	Responsabilidad
SimuladorApp	Extiende Application. Construye escena 1440×860 px con SplitPane de 3 columnas, barra de herramientas, tablas de cola, panel BCP, log y TabPane RAM/Disco.
SimuladorController	Implementa KernelEventListener. Expone ObservableList y StringProperty. Contiene ProcessRow, MemoryRow, DiskRow.

simuladorminipc.interrupt

Clase	Responsabilidad
InterruptManager	Cola FIFO con ArrayDeque. <code>raise()</code> , <code>poll()</code> , <code>clear()</code> , <code>snapshot()</code> .
Interrupt	Valor inmutable: tipo, PCB fuente, payload, intValue.
InterruptType	Enum de 9 tipos: PROCESS_FINISHED, QUANTUM_EXPIRED, IO_REQUEST, IO_FINISHED, PAGE_FAULT, KEYBOARD_INPUT, FILE_OPERATION, SCREEN_OUTPUT, RUNTIME_ERROR.

simuladorminipc.io

Clase	Responsabilidad
IOManager	3 instancias de Device + Map<Integer,PCB> de procesos en espera. submit/complete/tick/reset.
Device	TECLADO(MAX_VALUE), PANTALLA(1), DISCO(5) ticks de latencia. assign/tick/checkComplete/release.

simuladorminipc.kernel

Clase	Responsabilidad
Kernel	Propietario de todos los subsistemas. Pipeline de 10 etapas en executeTick(). Estado: booted, halted, waitingForKeyboard.

Clase	Responsabilidad
KernelEventListener	Interfaz Observer con 7 callbacks: onTickCompleted, onProcessStateChanged, onScreenOutput, onKeyboardInputRequired, onFileOperation, onExecutionError, onAllFinished.

simuladorminipc.memory

Clase	Responsabilidad
RAM	String[] de 700 celdas. OS_RESERVED=20. read/write/clearRange.
MemoryManager	Asignador first-fit. allocate/free/updateBcpOsArea/storeBcpInOsArea.
MemoryBlock	Descriptor: base, size, pid, free. getLimit().
MemoryConfig	Lee memory-config.json con regex. Fallback: RAM=512.
VirtualMemoryManager	translate(PCB, offset) = base+offset. isValid() valida rango.
SwapManager	Placeholder. swapOut/swapIn definidos pero no completamente implementados.

simuladorminipc.model

Clase	Responsabilidad
PCB	Bloque de Control de Proceso. PID atómico, 28 campos, instructions, registers, IO.
ProcessState	Enum 7 estados: NEW, READY, RUNNING, BLOCKED, SUSPENDED_READY, SUSPENDED_BLOCKED, TERMINATED.
RegisterSet	ac, ax, bx, cx, dx, ir, pc, ah, al, dxString, zeroFlag, stack[5], stackPointer.
IORequest	Solicitud E/S: Operation (READ/WRITE/CREATE/DELETE/KEYBOARD/SCREEN), countdown, complete.

simuladorminipc.process

Clase	Responsabilidad
ProcessManager	Admisión: MAX_CONCURRENT=5. loadArrivingProcesses(tick) intenta admitir nuevos cada tick.
QueueManager	5 colas: newQueue (LinkedList), readyQueue, blockedQueue, suspendedQueue, terminatedQueue (ArrayList).
StateManager	Transiciones legales con guardTransition(). admitToReady, setRunning, preemptToReady, blockOnIO, unblockToReady, terminate.

simuladorminipc.scheduler

Interfaz SchedulingAlgorithm con selectNextProcess(List<PCB>, PCB currentRunning) → PCB y getName() . En la operación actual del simulador, la política activa es FCFS.

Clase	Criterio	Apropiativo
FCFSScheduler	Menor arrivalTime (desempate: menor PID)	No

SchedulingPolicyManager conserva la infraestructura para otras políticas, pero la ejecución vigente se documenta con FCFS.

simuladorminipc.stats

Clase	Responsabilidad
StatisticsManager	totalTicks, activeTicks, finishedProcesses. Calcula utilización, throughput, tiempos promedio.
ProcessStats	Snapshot inmutable al terminar: PID, tiempos de inicio/fin/espera/retorno/respuesta/servicio, ratio.

simuladorminipc.storage

Clase	Responsabilidad
Disk	String[1080]. INDEX_SIZE=10. createFile/openFile/writeFile/readFile/deleteFile/storeProgram.
DiskFile	Descriptor: name, startAddress, size, content. setContent actualiza size.
DiskConfig	Lee disk-config.json . Configura diskSize y virtualMemorySize .

3. Clases Clave — Diagrama UML

Esta sección sintetiza la estructura orientada a objetos y el acoplamiento interno del sistema. La lectura recomendada es: clases individuales, relaciones de cardinalidad y, finalmente, especificaciones técnicas de consulta rápida.

Ver el diagrama completo en PlantUML: `documentacion/diagramas/diagrama_clases_completo.puml` . Los diagramas incluidos a continuacion son una vista didactica para lectura rapida en documento.

PCB — Bloque de Control de Proceso

Esta clase muestra el rol del BCP como contenedor de estado y contexto de ejecucion. Debe leerse de arriba hacia abajo: identidad y tiempos, memoria, registros y metodos de control de flujo.

```
classDiagram
    class PCB {
        +static AtomicInteger pidCounter
        +int pid
        +String name
        +ProcessState state
        +long arrivalTime
    }
```

```
- burstTime : long
- remainingTime : long
- executedTime : long
- priority : int
- waitingTime, turnaroundTime, responseTime : long
- memoryBase, memoryLimit, memoryRequired : int
- registers : RegisterSet
- instructions : List<Instruction>
- pendingIORequests : Queue<IORequest>
- startTime, endTime : long

+ getCurrentInstruction() : Instruction
+ hasMoreInstructions() : boolean
+ static resetPidCounter() : void
```

Kernel

Esta clase describe al `Kernel` como fachada central del sistema. Resalta la composicion de subsistemas y los puntos publicos de orquestacion por tick.

```
Kernel (simuladorminipc.kernel)

- clock : SystemClock
- processManager : ProcessManager
- queueManager : QueueManager
- stateManager : StateManager
- scheduler : SchedulingPolicyManager
- dispatcher : Dispatcher
- cpu : CPU
- ioManager : IOManager
- interruptManager : InterruptManager
- memoryManager : MemoryManager
- vmm : VirtualMemoryManager
- disk : Disk
- statisticsManager : StatisticsManager
- listeners : List<KernelEventListener>
- booted, halted : boolean
- waitingForKeyboard : PCB

+ boot() : void
+ executeTick() : void
+ loadProcess(PCB) : void
+ provideKeyboardInput(int) : void
+ reset() : void
+ setPolicy(Policy) : void
+ setQuantum(int) : void
+ addListener(KernelEventListener) : void
```

CPU

El diagrama UML de `CPU` resume el ciclo de ejecucion y su dependencia de memoria virtual y disco. Permite ubicar rapidamente donde se transforma una instruccion en un `CycleEvent` .

CPU (simuladorminipc.cpu)
- id : int - currentProcess : PCB - cycleCountdown : int - disk : Disk - vmm : VirtualMemoryManager
+ executeCycle() : CycleEvent - applyInstruction(Instruction) : CycleEvent + loadProcess(PCB) : void + releaseProcess() : void + isIdle() : boolean + getCurrentProcess() : PCB

Dispatcher

Esta clase separa claramente la responsabilidad de cambio de contexto respecto de la logica del planificador, lo cual facilita medir y auditar los *context switches*.

Dispatcher (simuladorminipc.cpu)
- contextSwitches : int
+ dispatch(PCB, CPU) : void + preempt(CPU) : void + getContextSwitches() : int + reset() : void

SimuladorController

Esta clase muestra el puente entre UI y modelo: el controlador observa eventos del kernel, mantiene colecciones observables y sincroniza la representacion visual del estado interno.

SimuladorController (fx) implements KernelEventListener
- kernel : Kernel - autoTimeline : Timeline (400 ms) - newQueueRows, readyQueueRows, blockedQueueRows, terminatedQueueRows : ObservableList<ProcessRow> - memoryRows : ObservableList<MemoryRow> - diskRows : ObservableList<DiskRow> - currentTickProp : LongProperty - cpuStatusProp, cpuProcessProp, reg*Prop : StringProperty - waitingForKeyboardProcess : PCB
+ loadFiles() : void + step() : void + toggleAuto() : void + startSingleProcess() : void + reset() : void

```
| + provideKeyboardInput(int) : void |
| + refreshUI() : void |
| [KernelEventListener] onTickCompleted, onProcessStateChanged, |
| onScreenOutput, onKeyboardInputRequired, onAllFinished, etc. |
```

3.1 Acoplamiento UML de Clases (Cardinalidad)

Diagrama UML de relaciones entre clases con cardinalidad. Se centra en composicion del `Kernel` , asociaciones de ejecucion (CPU-PCB) y dependencias de infraestructura (RAM, disco, listeners).

```
SimuladorController "1" — "1" Kernel : controla
Kernel "1" *— "1" CPU : compone
Kernel "1" *— "1" Dispatcher : compone
Kernel "1" *— "1" SchedulingPolicyManager : compone
Kernel "1" *— "1" ProcessManager : compone
Kernel "1" *— "1" QueueManager : compone
Kernel "1" *— "1" StateManager : compone
Kernel "1" *— "1" MemoryManager : compone
Kernel "1" *— "1" VirtualMemoryManager : compone
Kernel "1" *— "1" Disk : compone
Kernel "1" *— "1" IOManager : compone
Kernel "1" *— "1" StatisticsManager : compone

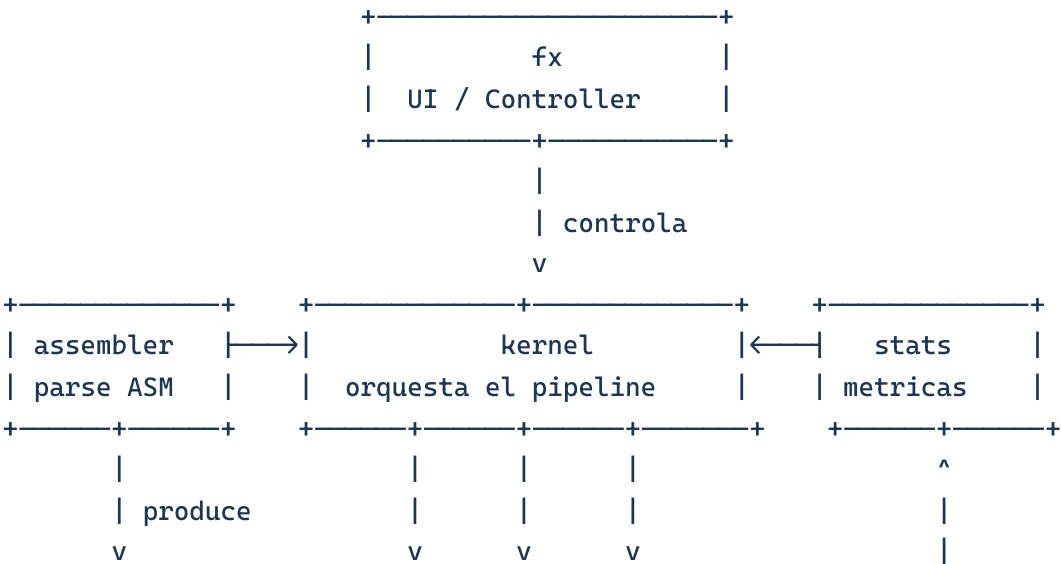
CPU "1" — "0..1" PCB : ejecuta
QueueManager "1" *— "0..*" PCB : organiza colas
Disk "1" *— "0..*" DiskFile : almacena archivos
MemoryManager "1" *— "1" RAM : administra
VirtualMemoryManager "1" — "1" SwapManager : respalda
Kernel "1" — "0..*" KernelEventListener : notifica
```

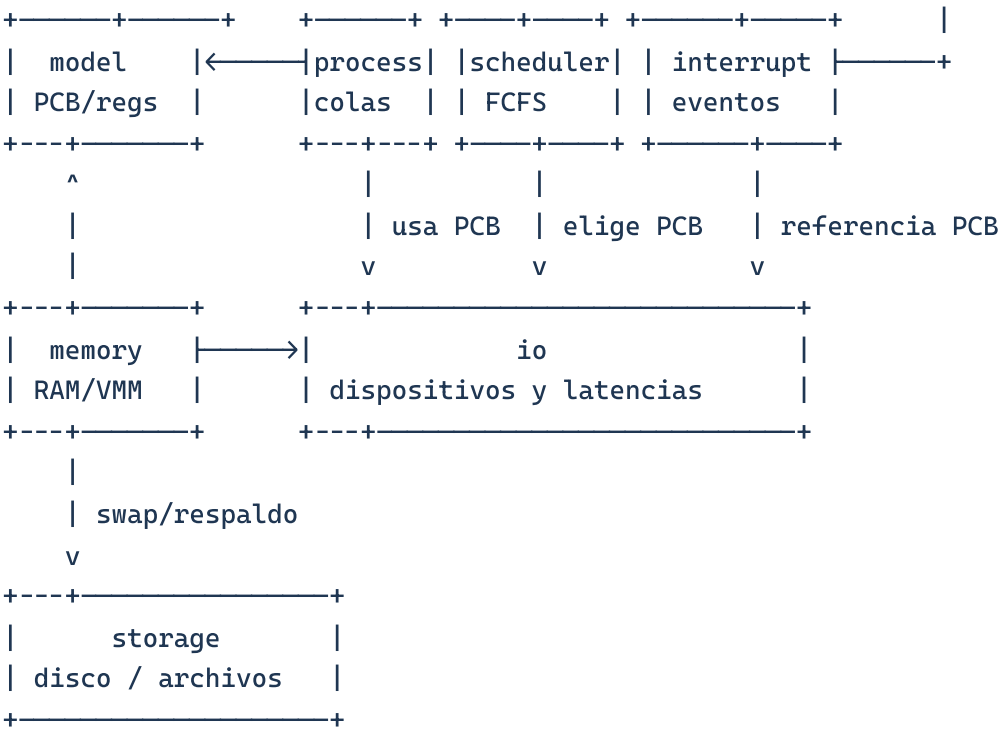
3.2 Acoplamiento UML de Paquetes (Cardinalidad)

En esta subseccion se incluyen dos diagramas UML de paquetes para que la arquitectura sea visible y explicativa: una vista estructural y una vista de flujo de dependencias. Ambos diagramas estan pensados para lectura didactica.

3.2.1 Diagrama UML de Paquetes — Vista Estructural

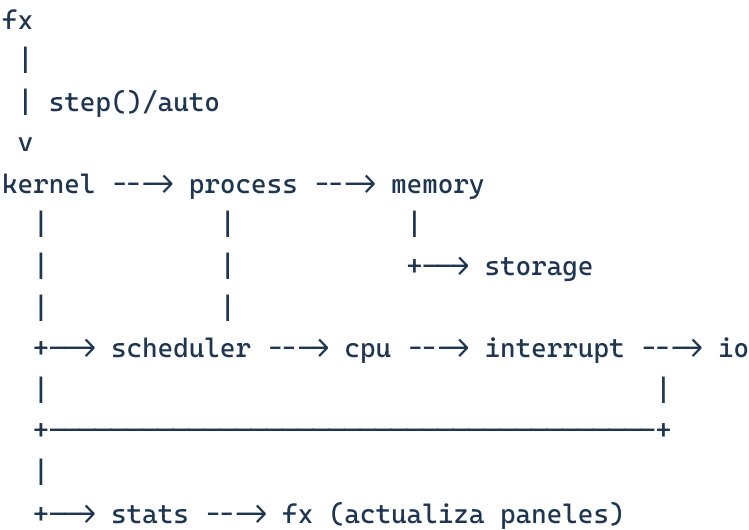
Esta vista ubica el paquete `kernel` como nucleo y muestra los paquetes de soporte que lo rodean. Las flechas indican dependencias de uso y las etiquetas muestran la intencion de cada acoplamiento.



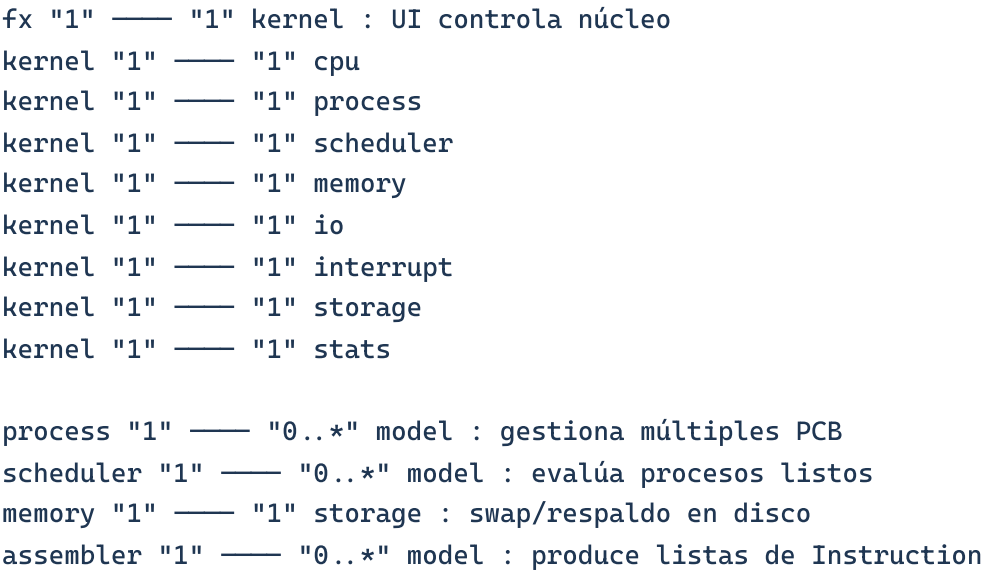


3.2.2 Diagrama UML de Paquetes — Flujo Operativo por Tick

Esta vista enfatiza el recorrido funcional durante un tick: admision, seleccion, ejecucion, interrupcion, E/S y retroalimentacion de metricas hacia la interfaz.



Nota*: A --> B significa que A depende de B para completar su responsabilidad en el ciclo de ejecucion.



interrupt "1" — "0..*" model : referencia proceso origen

3.3 Especificación de Clases

La siguiente sección documenta objetivo, atributos, constructores y métodos clave para facilitar mantenimiento y trazabilidad.

Clase: `PCB` (`simuladorminipc.model`)

```
/**
 * Objetivo:
 * Representar el Bloque de Control de Proceso con toda la información
 * necesaria para planificación, ejecución, memoria y estadísticas.
 *
 * Atributos principales:
 * @field pid          Identificador único del proceso.
 * @field state        Estado actual del ciclo de vida del proceso.
 * @field instructions Lista inmutable/lógica de instrucciones ASM.
 * @field registers    Snapshot de registros de CPU del proceso.
 * @field memoryBase/limit Rango físico asignado en RAM.
 *
 * Métodos clave:
 * @method getCurrentInstruction()
 * @return Instruction instrucción actual según PC.
 * @method hasMoreInstructions()
 * @return boolean true si el proceso puede seguir ejecutando.
 */
```

Clase: `Kernel` (`simuladorminipc.kernel`)

```
/**
 * Objetivo:
 * Orquestrar todos los subsistemas del simulador y ejecutar el pipeline
 * de 10 etapas por tick.
 *
 * Atributos principales:
 * @field cpu, dispatcher, policyManager  Núcleo de ejecución y planificación.
 * @field queueManager, stateManager      Gestión de colas y transiciones.
 * @field memoryManager, vmManager, disk  Gestión de memoria y almacenamiento.
 * @field ioManager, interruptManager     Interrupciones y E/S.
 *
 * Constructores:
 * @constructor Kernel(int ramSize, int diskSize)
 * @param ramSize tamaño de RAM simulada en celdas.
 * @param diskSize tamaño de disco simulado en celdas.
 *
 * Métodos clave:
 * @method boot() Inicializa reloj, banderas y estado del kernel.
 * @method executeTick() Ejecuta un ciclo completo del SO simulado.
 * @method loadProgram(File file, int arrivalTime, int priority)
 * @param file archivo asm a ensamblar y cargar.
 * @param arrivalTime tick de llegada del proceso.
 * @param priority prioridad inicial (numérica).
 */
```

Clase: CPU (simuladorminipc.cpu)

```
/**
 * Objetivo:
 *   Ejecutar instrucciones del proceso activo y producir eventos de ciclo.
 *
 * Atributos principales:
 *   @field currentProcess  PCB cargado actualmente en CPU.
 *   @field cycleCountdown  Ticks restantes para finalizar instrucción pesada.
 *
 * Métodos clave:
 *   @method executeCycle()
 *     @return CycleEvent resultado del ciclo (normal, I/O, finalización, error).
 *   @method loadProcess(PCB p)
 *     @param p proceso a cargar en CPU.
 *   @method releaseProcess()
 *     @return PCB proceso desalojado/liberado de CPU.
 */
```

Clase: SimuladorController (simuladorminipc.fx)

```
/**
 * Objetivo:
 *   Actuar como controlador/presenter entre Kernel y vista JavaFX.
 *
 * Atributos principales:
 *   @field kernel          Fachada principal del modelo.
 *   @field autoTimeline    Temporizador para autoejecución.
 *   @field *Rows           Colecciones observables para tablas de UI.
 *   @field stat*           Propiedades de métricas para binding visual.
 *
 * Constructor:
 *   @constructor SimuladorController()
 *     Carga MemoryConfig + DiskConfig, crea Kernel y registra listeners.
 *
 * Métodos clave:
 *   @method loadFiles(List<File> files)
 *     @param files archivos ASM seleccionados por usuario.
 *     @return List<String> errores de parseo/carga.
 *   @method step() avanza un tick y refresca paneles.
 *   @method reset() limpia estado completo de simulación.
 */
```

Clase: MemoryConfig (simuladorminipc.memory)

```
/**
 * Objetivo:
 *   Cargar configuración de RAM desde memory-config.json.
 *
 * Atributos:
 *   @field ramSize tamaño total de memoria principal (celdas).
 *
 * Métodos:
 *   @method load()  Carga config desde classpath con fallback seguro.
 *   @method defaults() Devuelve valores por defecto del subsistema.
```

*/

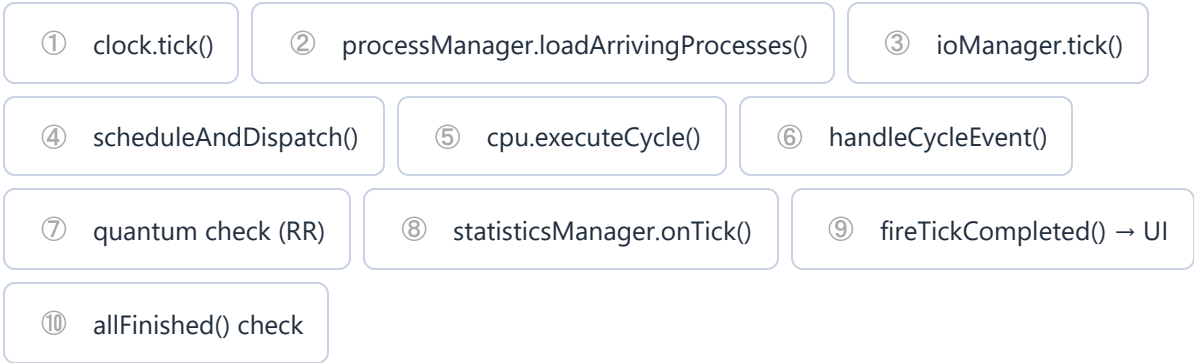
Clase: `DiskConfig` (`simuladorminipc.storage`)

```
/**
 * Objetivo:
 *   Cargar configuración de disco y memoria virtual desde disk-config.json.
 *
 * Atributos:
 *   @field diskSize          tamaño del almacenamiento secundario.
 *   @field virtualMemorySize tamaño de espacio virtual por proceso.
 *
 * Métodos:
 *   @method load() carga valores configurables del classpath.
 *   @method defaults() valores de respaldo cuando no hay archivo válido.
 */
```

4. Pipeline de Ejecución

Esta sección explica, de forma secuencial, cómo se ejecuta el sistema operativo simulado en tiempo discreto. Cada tick mantiene un orden estable para garantizar reproducibilidad, depuración y consistencia métrica.

4.1 Ciclo Tick — 10 Etapas



El método `executeTick()` siempre sigue este orden fijo. La idea es: primero se actualiza el tiempo y el estado interno del sistema, luego se ejecuta CPU, y al final se publican resultados y se verifica el cierre de la simulación.

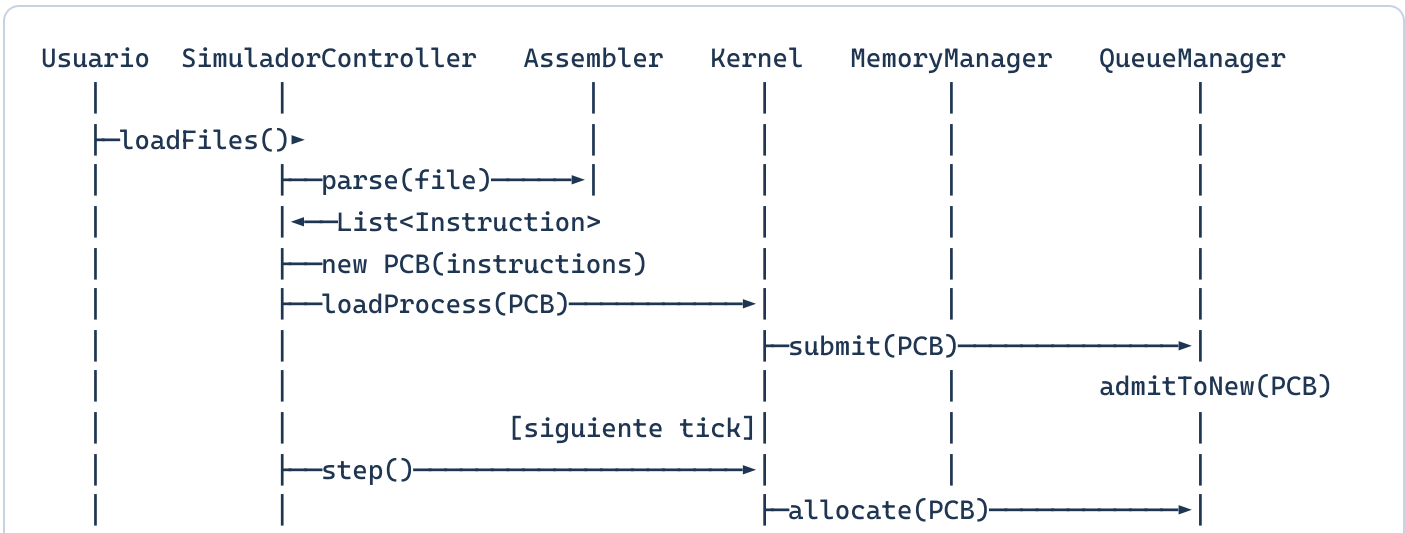
Etapa	Qué hace	Resultado inmediato
① <code>clock.tick()</code>	Incrementa el reloj global del sistema.	El tick actual avanza en 1; todas las decisiones del ciclo usan este nuevo valor.
② <code>processManager.loadArrivingProcesses()</code>	Revisa procesos en <code>NEW</code> cuyo tiempo de llegada ya se cumplió e intenta admitirlos.	Si hay RAM disponible pasan a <code>READY</code> ; si no, permanecen en <code>NEW</code> .

Etapas	Qué hace	Resultado inmediato
③ <code>ioManager.tick()</code>	Avanza los contadores de latencia de dispositivos de E/S.	Los procesos con E/S finalizada se desbloquean y regresan a <code>READY</code> .
④ <code>scheduleAndDispatch()</code>	El planificador elige el siguiente proceso elegible y el dispatcher realiza el cambio de contexto.	La CPU queda cargada con el proceso que correrá en este tick (si existe alguno).
⑤ <code>cpu.executeCycle()</code>	Ejecuta una unidad de trabajo de CPU sobre el proceso actual.	Devuelve un <code>CycleEvent</code> que resume lo ocurrido en la ejecución.
⑥ <code>handleCycleEvent()</code>	Interpreta el evento de CPU y aplica transiciones de estado o interrupciones según corresponda.	Se procesa el caso: normal, finalización, error, solicitud de E/S, teclado o salida a pantalla.
⑦ Quantum check (extensión futura)	Reservado para políticas con quantum. En FCFS activo esta verificación no produce desalojo.	Sin efecto operativo en FCFS; queda como punto de extensión del pipeline.
⑧ <code>statisticsManager.onTick()</code>	Actualiza métricas globales del sistema y persiste el BCP en área del SO.	Se actualizan <code>totalTicks</code> , <code>activeTicks</code> y snapshots de estado para análisis.
⑨ <code>fireTickCompleted()</code>	Notifica a los listeners (UI/controlador) que el tick terminó.	La interfaz se refresca con colas, CPU, memoria, disco y registros actualizados.
⑩ <code>allFinished()</code> check	Comprueba si ya no quedan procesos por ejecutar.	Si todo terminó, el kernel marca <code>halted=true</code> y dispara <code>onAllFinished()</code> .

Resumen práctico: cada tick ejecuta exactamente una vuelta completa del sistema operativo simulado: admitir, desbloquear, planificar, ejecutar, registrar y notificar.

4.2 Diagrama de Secuencia — Carga de Archivo

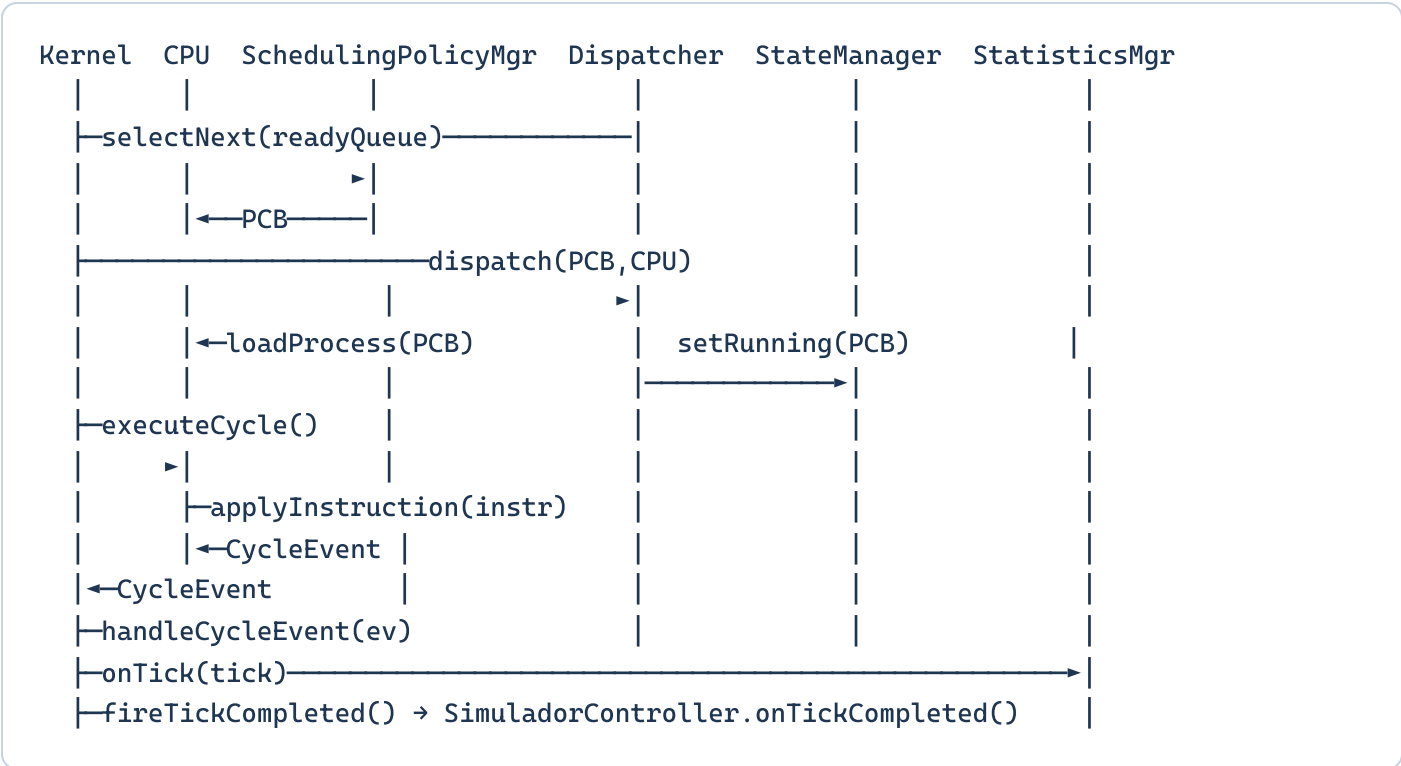
Diagrama UML de secuencia que describe el caso de uso de carga: desde la acción del usuario, pasando por ensamblado, creación de PCB y admisión efectiva al sistema.





4.3 Diagrama de Secuencia — Tick Completo

Diagrama UML de secuencia para un tick de ejecucion: seleccion del proceso, despacho, ejecucion en CPU, manejo del evento y actualizacion de metricas/UI.



5. Gestión de Memoria

Esta sección detalla la asignación de memoria principal, la validación del direccionamiento virtual y los parámetros configurables para ejecutar escenarios con diferentes perfiles de carga.

5.1 Algoritmo First-Fit

Estructura de RAM (700 celdas):
Celdas 0–19 → Área del SO (BCPs de procesos activos)
Celdas 20–699 → Área de usuario (instrucciones de procesos)

Al admitir proceso P con memoryRequired = N:
Para cada MemoryBlock b (ascendente por dirección):
 si b.free && b.size ≥ N:
 b.free = false; b.pid = P.pid
 P.memoryBase = b.base; P.memoryLimit = b.base + b.size - 1
 Escribe instrucciones en RAM[b.base .. b.base+N-1]
 return true
return false ← proceso permanece en NEW

5.2 Traducción de Memoria Virtual

```
Dirección física = P.memoryBase + virtualOffset

Validación:
    si virtualOffset < 0 || virtualOffset ≥ (P.memoryLimit - P.memoryBase):
        → CycleEvent.error("Acceso fuera de rango de memoria")
```

Parámetro	Valor actual	Configurable en
ramSize	700	memory-config.json
virtualMemorySize	80	disk-config.json
diskSize	1080	disk-config.json
maxInstructions	80	assembler-config.json

6. Interrupciones y E/S

Aquí se resume el modelo de interrupciones y su interacción con los dispositivos de entrada/salida, incluyendo latencias operativas y su impacto directo sobre el ciclo de estados de los procesos.

Latencias de dispositivos

Dispositivo	Latencia	Nota
Teclado	Integer.MAX_VALUE	No completa automáticamente; requiere provideKeyboardInput()
Pantalla	1 tick	No bloquea al proceso en la práctica
Disco	5 ticks	Bloquea el proceso 5 ticks completos

Tipos de interrupción (InterruptType)

PROCESS_FINISHED	← INT 20H o PC fuera de rango
QUANTUM_EXPIRED	← reservado para políticas con quantum (no activo en FCFS)
IO_REQUEST	← INT 09H o INT 21H
IO_FINISHED	← device.checkComplete() == true
PAGE_FAULT	← acceso a página no cargada (extensión futura)
KEYBOARD_INPUT	← usuario envía valor por campo de teclado
FILE_OPERATION	← subcódigo de disco (INT 21H)
SCREEN_OUTPUT	← INT 10H
RUNTIME_ERROR	← stack overflow, underflow, instrucción inválida

Flujo de solicitud de disco (INT 21H)

```
Tick N:    INT 21H → CycleEvent.fileOperation(ah, dxString)
           ioManager.submitFileRequest(P, subcode, filename)
           stateManager.blockOnIO(P) → BLOCKED
```

```
Tick N+5: device.checkComplete() == true
          stateManager.unblockToReady(P) → READY
```

7. Manejo de Errores

Esta sección clasifica los errores en dos momentos del ciclo de vida: carga (parseo) y ejecución (runtime). El objetivo es evidenciar cómo se aísla cada falla sin comprometer la continuidad de la simulación global.

Errores de parse (carga)

Error	Excepción	Efecto
Opcode desconocido	Exception("[Línea N] Instrucción desconocida: 'X'")	Archivo rechazado; demás continúan
Operandos incorrectos	Exception("[Línea N] Se esperaban N operandos ... ")	Ídem
Registro inválido	Exception("[Línea N] Registro inválido 'EAX' ... ")	Ídem
Literal en registro no-DX	Exception("[Línea N] Los literales solo pueden ... ")	Ídem
Archivo vacío	Exception("El archivo está vacío ... ")	Ídem
Excede maxInstructions	Exception("El programa excede el límite ... ")	Ídem

Errores de runtime (ejecución)

Error	Causa	Efecto
Stack overflow	6.º PUSH (pila llena cap=5)	Proceso termina; memoria liberada; demás continúan
Stack underflow	POP con pila vacía	Ídem
Instrucción no manejada	InstructionType sin caso en switch	Ídem — indicaría bug interno

8. Métricas de Rendimiento

Esta sección define los indicadores cuantitativos del simulador y las fórmulas empleadas para evaluar eficiencia de CPU, capacidad de procesamiento y calidad temporal de atención a procesos.

Métrica	Fórmula
Utilización de CPU (%)	$\text{activeTicks} / \text{totalTicks} \times 100$
Throughput	$n_{\text{terminados}} / \text{totalTicks}$
Tiempo de espera promedio	$\Sigma \text{waitingTime}_i / n$
Tiempo de retorno promedio	$\Sigma \text{turnaroundTime}_i / n$
Tiempo de respuesta promedio	$\Sigma \text{responseTime}_i / n$
Ratio de retorno	$\text{turnaroundTime}_i / \text{burstTime}_i$

Definiciones:

- `turnaroundTime` = `endTime` - `arrivalTime`
- `responseTime` = `startTime` - `arrivalTime` (primer tick en CPU)
- `waitingTime` = ticks acumulados en cola READY
- `serviceTime` = ticks reales de CPU ejecutados

9. Conjunto de Instrucciones ASM

Esta sección funciona como referencia operativa del lenguaje ASM del simulador: sintaxis, costo por instrucción y efectos esperados sobre registros, memoria e interrupciones.

Instrucción	Sintaxis	Peso	Descripción
MOV	MOV dest, src	1	Copia valor entre registros o asigna literal
LOAD	LOAD AC, offset	2	Carga RAM[base+offset] en AC
STORE	STORE offset, AC	2	Escribe AC en RAM[base+offset]
ADD	ADD dest, src	3	<code>dest = dest + src</code>
SUB	SUB dest, src	3	<code>dest = dest - src</code>
INC	INC reg	1	<code>reg = reg + 1</code>
DEC	DEC reg	1	<code>reg = reg - 1</code>
SWAP	SWAP r1, r2	1	Intercambia dos registros
CMP	CMP r1, r2	2	Activa <code>zeroFlag</code> si <code>r1 == r2</code>
JMP	JMP N	2	Salto incondicional a instrucción N
JE	JE N	2	Salta si <code>zeroFlag == true</code>
JNE	JNE N	2	Salta si <code>zeroFlag == false</code>
PUSH	PUSH reg	1	Apila valor (máx. 5)

Instrucción	Sintaxis	Peso	Descripción
POP	POP reg	1	Desapila en registro
PARAM	PARAM val	3	Apila literal entero
INT 20H	INT 20H	2	Termina el proceso
INT 10H	INT 10H	2	Salida a pantalla (AC o DX)
INT 09H	INT 09H	3	Entrada de teclado — bloquea
INT 21H	INT 21H	5	Operación de archivo (AH=subcode, DX=nombre)

Registros válidos: AC , AX , BX , CX , DX , AH , AL

10. Estados del Proceso

Esta sección describe el autómata de estados del proceso y las condiciones que activan cada transición, permitiendo correlacionar eventos de CPU/E/S con cambios observables en las colas del sistema.

NEW
→
READY
→
RUNNING
→
BLOCKED
→
READY
→ ... →
TERMINATED

Transición	Condición	Método
NEW → READY	arrivalTime ≤ tick, RAM disponible, concurrentes < 5	admitToReady()
READY → RUNNING	Seleccionado por planificador y CPU ociosa	setRunning()
RUNNING → READY	No aplica en FCFS actual (sin desalojo por quantum)	preemptToReady()
RUNNING → BLOCKED	INT 09H o INT 21H	blockOnIO()
BLOCKED → READY	E/S completada o teclado recibido	unblockToReady()
* → TERMINATED	INT 20H, PC ≥ instrucciones.size(), error de runtime	terminate()

11. Manual de Usuario

Este manual describe la operación funcional del simulador desde la perspectiva del usuario final, con una guía progresiva: inicio, flujo recomendado, lectura de paneles y resolución de incidencias.

11.1 Inicio rápido

- Abra el proyecto `simuladorMiniPC` en NetBeans.
- Ejecute `F6` para iniciar la aplicación.
- Pulse `Cargar archivos` y seleccione programas `.asm`.
- Use `Paso` para ejecución detallada o `Auto-todos` para ejecución continua.

11.2 Flujo operativo recomendado

- Cargar 2-3 programas ASM válidos para observar interacción entre colas.
- Verificar transición de NEW a READY tras admisión en memoria.
- Revisar eventos de pantalla/teclado/disco en bitácora.
- Al terminar, usar panel de métricas para analizar rendimiento.

11.3 Guía de interpretación de paneles

Panel	Qué observar	Cuándo usarlo
BCP en CPU	Registros, PC/IR, burst restante, estado RUNNING	Depuración instrucción por instrucción
Colas de procesos	Distribución NEW/READY/BLOCKED/TERMINATED	Análisis del ciclo de vida y bloqueo
RAM	Asignación por PID (identificador de proceso) y zona reservada del SO	Validar admisión y first-fit
Disco	Índice de archivos y celdas ocupadas	Comprobar operaciones INT 21H
Bitácora	Eventos [LOAD], [SCHED], [I/O], [ERROR]	Trazabilidad temporal de ejecución

11.4 Manejo de incidencias frecuentes

- Archivo ASM rechazado: revisar línea reportada y sintaxis de operandos.
- Proceso no avanza: verificar si está BLOCKED por teclado o disco.
- Entrada de teclado pendiente: proveer valor cuando aparezca solicitud INT 09H.
- Resultados inesperados: reiniciar simulación y repetir en modo paso a paso.

12. Manual Técnico

Este manual consolida decisiones de diseño, acoplamiento, configuración y mantenimiento. Su finalidad es facilitar la continuidad técnica del proyecto y reducir riesgos durante cambios evolutivos.

12.1 Arquitectura y responsabilidades

- **fx**: UI JavaFX, binding de propiedades, orquestación de interacción.
- **kernel**: ejecución del pipeline por tick y coordinación de subsistemas.
- **memory**: RAM física, asignación first-fit, validación base+offset.
- **storage**: persistencia simulada de archivos y respaldo de memoria virtual.
- **scheduler**: interfaz Strategy, con FCFS operativo en la versión actual.

12.2 Configuración externa (JSON)

Archivo	Parámetros	Subsistema consumidor
memory-config.json	ramSize	MemoryConfig / Kernel
disk-config.json	diskSize , virtualMemorySize	DiskConfig / VirtualMemoryManager
assembler-config.json	maxInstructions	AssemblerConfig / Assembler

13. Objetivos Alcanzados

Esta sección sintetiza los objetivos logrados durante el desarrollo, alineados con los resultados esperados de la fase de proyecto académico (Proyecto 1): modelado de componentes de un sistema operativo, ejecución por ticks, gestión de procesos y evaluación comparativa de políticas de planificación.

Objetivo académico	Evidencia de cumplimiento en el simulador
Modelar la arquitectura básica de un SO	Núcleo con subsistemas separados (CPU, memoria, scheduler, E/S, interrupciones, estadísticas, almacenamiento) y coordinación central en <code>Kernel</code> .
Representar el ciclo de vida de procesos	Implementación de 7 estados (NEW, READY, RUNNING, BLOCKED, SUSPENDED_READY, SUSPENDED_BLOCKED, TERMINATED) y transiciones controladas por <code>StateManager</code> .
Implementar planificación de CPU	FCFS activo en la operación actual, con interfaz común <code>SchedulingAlgorithm</code> preparada para habilitar otras políticas.
Integrar gestión de memoria principal y virtual	Asignación first-fit, reserva del área del SO (0-19), traducción base+offset y validación de acceso por proceso con <code>VirtualMemoryManager</code> .
Simular interrupciones y E/S con latencia	INT 09H, INT 10H, INT 20H, INT 21H; dispositivos con latencias diferenciadas (teclado, pantalla, disco) y desbloqueo por finalización.
Construir una interfaz de observación del estado interno	UI JavaFX 1440x860 con colas, RAM/Disco, BCP activo, registros y bitácora de eventos en tiempo real.
Medir desempeño del sistema	Cálculo de utilización de CPU, throughput, tiempos promedio de espera/retorno/respuesta y estadísticas por proceso en <code>StatisticsManager</code> .
Validar robustez frente a errores	Manejo explícito de errores de parse y runtime (stack overflow/underflow, operandos inválidos, límites de instrucción, etc.) sin colapsar la simulación global.

14. Bibliografía

Esta sección documenta las fuentes teóricas que sustentan las decisiones de arquitectura, planificación, manejo de procesos, memoria y prácticas de ingeniería de software aplicadas en el simulador.

Referencias teóricas empleadas para el diseño de los componentes de planificación, procesos, memoria y arquitectura:

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.
2. Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson.
3. Stallings, W. (2018). *Operating Systems: Internals and Design Principles* (9th ed.). Pearson.
4. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
5. Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson.
6. Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill.